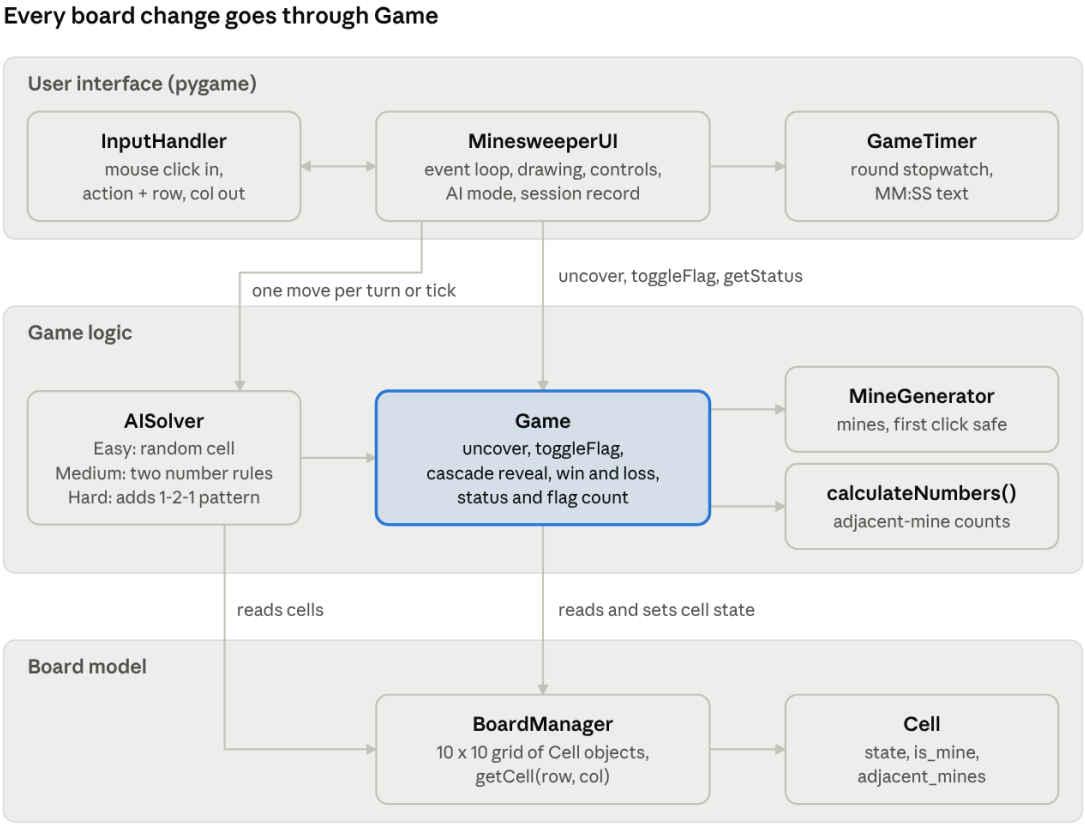
Minesweeper System Architecture Overview

Oct 11, 2026 · Team 2

The game is a Python and pygame application in three layers: a UI that owns the window and the event loop, a game-logic layer that owns the rules, and a board model that owns the data. An AI solver sits beside the UI and plays through the same two game-logic calls a human click uses.

System components

Ten modules in `src/` make up the system, and every change to the board goes through the `Game` class. The UI and the AI solver only read cells; they never set a cell's state themselves.



component diagram · 9 modules in 3 layers

Arrows show which module calls which. The UI also reads cells from `BoardManager` directly when it draws the board. The arrow from `MinesweeperUI` to `BoardManager` is left out to keep the picture readable.

Module	Main class or function	Responsibility	Uses

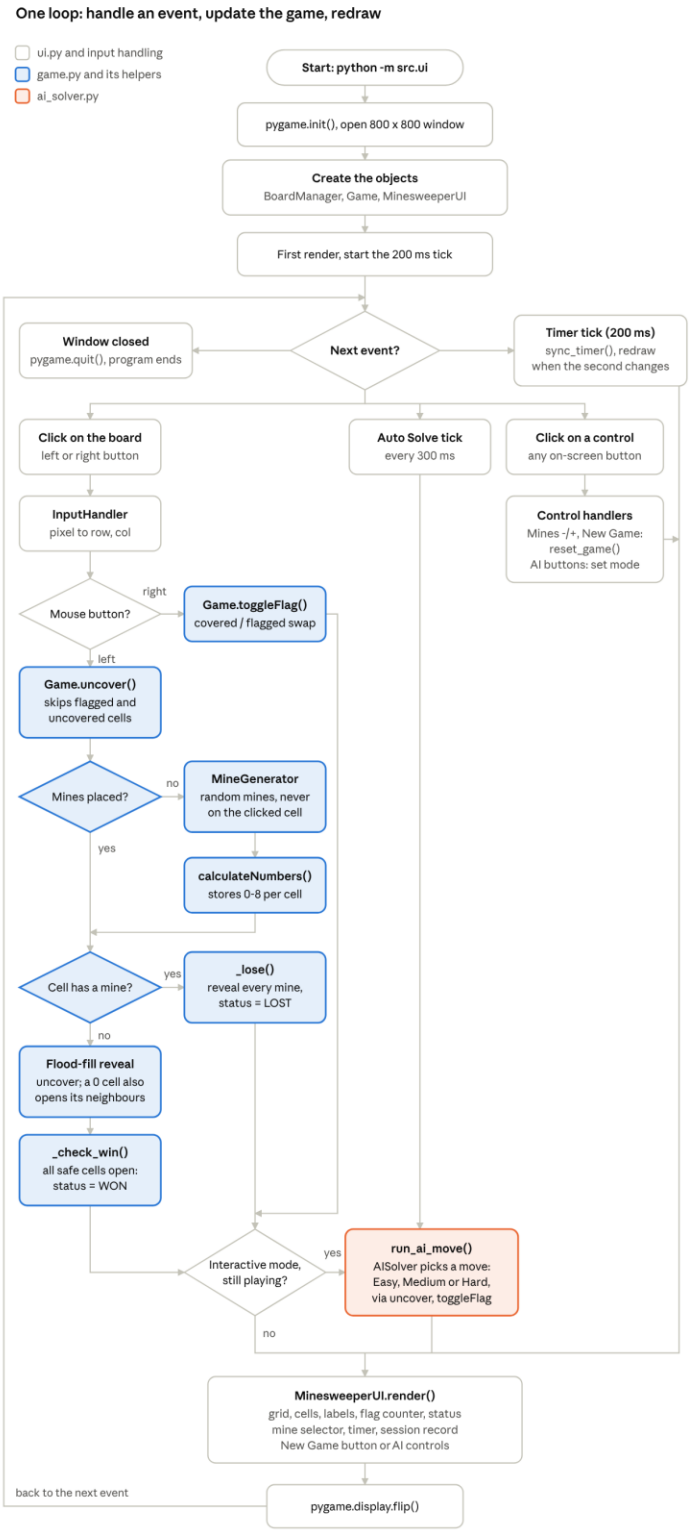
Module	Main class or function	Responsibility	Uses
`ui.py`	`MinesweeperUI`, `main()`	Opens the window, runs the event loop, draws everything, and handles the on-screen controls (mine count, AI difficulty and mode, New Game, timer, session record). This is the program's entry point.	`Game`, `BoardManager`, `InputHandler`, `AISolver`, `GameTimer`
`input_handler.py`	`InputHandler`	Turns a mouse click's pixel position into a board action: uncover or flag, plus row and column. Ignores clicks off the board.	pygame events only
`game_timer.py`	`GameTimer`, `format_time()`	Stopwatch for one round, and MM:SS formatting. Has no pygame code.	nothing
`ai_solver.py`	`AISolver`	Makes one AI move per call at Easy, Medium or Hard difficulty.	`Game`, `BoardManager`
`game.py`	`Game`, `GameStatus`	The rules: uncover, flag, first-click mine placement, cascade reveal, win and loss.	`BoardManager`, `Cell`, `MineGenerator`, `calculateNumbers`
`mine_generator.py`	`MineGenerator`	Picks random mine positions, never the first-clicked cell.	`BoardManager`
`number_calculator.py`	`calculateNumbers()`	Counts adjacent mines for every safe cell and stores the result in the cell.	`BoardManager`
`board.py`	`BoardManager`	Holds the 10 x 10 grid and gives access to cells by row and column.	`Cell`

Module	Main class or function	Responsibility	Uses
`cell.py`	`Cell`	One square: its visible state, whether it has a mine, and its adjacent-mine count.	nothing
`main.py`	`main()`	Builds a board and a game and prints setup text. It does not open the window; the game is started with `python -m src.ui`.	`Game`, `BoardManager`

Dependencies point one way, from the UI down to the board model. Nothing in the game-logic or model layers imports pygame, which is why those layers can be unit tested without a display.

Program flow

The program is a single loop: take the next event, let it change the game, and redraw. A human click and an AI move both end in the same two `Game` calls.



program flowchart · main loop, 5 event types, 4 decisions

Five kinds of event enter the loop. Closing the window ends the program. The other four all lead to a redraw and back to the loop. The loop has no frame-rate limit because it polls pygame's event queue continuously, and the two timers (200 ms for the clock, 300 ms for Auto Solve) supply events when the player is idle.

Data flow

Data moves in one direction: input goes down into `Game`, `Game` changes the cells, and the UI reads the cells back on the next redraw. The UI is never told what changed. It redraws the whole screen from current state.

1. **Event in.** pygame delivers a mouse click to `MinesweeperUI.handle_event()`. Clicks on the controls are handled there. Any other click is passed to `InputHandler`.
2. **Pixel to action.** `InputHandler` subtracts the board origin (200, 200), divides by the 40-pixel cell size, and returns `{"action": "uncover" or "flag", "row": r, "col": c}`, or `None` for a click that does not target a cell on the board.
3. **Action to rule.** The UI calls `Game.uncover(row, col)` or `Game.toggleFlag(row, col)`.
4. **First uncover only.** `Game` asks `MineGenerator` to place the mines, skipping the clicked cell, then `calculateNumbers()` writes a 0-8 count into every safe cell.
5. **Board change.** `Game` sets cell states. A mine ends the game as a loss and reveals every mine. A safe cell is uncovered, and a cell with a count of 0 cascades to its neighbours. `Game` then checks for a win.
6. **AI turn, if any.** In Interactive mode the UI calls `AISolver` once after the player's action. In Auto Solve mode a 300 ms timer event calls it instead. The solver reads the cells and calls the same `uncover` and `toggleFlag` methods.
7. **State out.** `render()` reads each cell's state from `BoardManager`, the status and remaining-flag count from `Game`, and the elapsed time from `GameTimer`, then draws the frame.

Nothing is written to the disk. All states, including the session's record time, lives in memory and is lost when the window closes.

Key data structures

The whole game state is a 10 x 10 grid of `Cell` objects plus a handful of counters that are tracked in `Game`. Everything else is either derived from those or belongs to the UI.

Structure	Where	Shape	Meaning
`Cell`	`cell.py`	`state` (int), `is_mine` (bool), `adjacent_mines` (int 0-8)	One square. `state` is `COVERED` = 0, `FLAGGED` = 1 or `UNCOVERED` = 2.
`BoardManager.grid`	`board.py`	List of 10 rows, each a list of 10 `Cell` objects, indexed `grid[row][col]`	The board. Row 0 is the top row (labelled 1); column 0 is the left column (labelled A).

Structure	Where	Shape	Meaning
`GameStatus`	`game.py`	String enum: `IN_PROGRESS`, `WON`, `LOST`	The round's state. Because it is a string enum, the UI compares it with plain strings such as `"WON"`.
`Game` counters	`game.py`	`mine_count`, `_mines_placed`, `_revealed_count`, `_flags_placed`, `_total_safe_cells`	Win is `_revealed_count >= _total_safe_cells`. Remaining flags is `mine_count - _flags_placed`.
Mine set	`mine_generator.py`	Set of `(row, col)` tuples	The positions chosen by `random.sample()` from the 99 cells other than the first click.
Flood-fill stack	`game.py`	List of `(row, col)` used as a stack, plus a `visited` set	Drives the cascade reveal without recursion.
Action dict	`input_handler.py`	`{"action", "row", "col"}`	The only message passed from input handling to the UI.
AI working lists	`ai_solver.py`	Lists and sets of `(row, col)` built fresh on each move	Hidden and flagged neighbours of each numbered cell. The solver keeps no memory between moves.
`GameTimer`	`game_timer.py`	`_start_time` (float or `None`), `_elapsed` (float)	Running when `_start_time` is set; reports whole seconds, capped at 99:59.
UI state	`ui.py`	`ai_difficulty`, `ai_mode`, `auto_running`, `mine_count`, `record_seconds`, `record_difficulty`	Player settings and the session record. All but auto_running survive New Game; the board and `Game` are rebuilt.
Layout constants	`ui.py`, `game.py`	`BOARD_ORIGIN` (200,	Screen geometry and

Structure	Where	Shape	Meaning
		200), `CELL_SIZE` 40, `MIN_MINES` 10, `MAX_MINES` 20	the allowed mine range.

Coordinates are always `(row, col)` with zero-based indexes, in every module.

Extension points

Most additions touch one layer only since the UI, the rules, and the data are separate. The table lists where each kind of change goes.

To add or change	Edit	What to know
A new AI difficulty	Add a `*_move()` method to `AISolver`; add a branch in `MinesweeperUI.run_ai_move()` and a button in `draw_ai_controls()`	The method only needs to call `game.uncover()` or `game.toggleFlag()` and return the status.
A new game rule, such as uncovering all neighbours of a satisfied number	Add a method to `Game`; add an action for it in `InputHandler` and a branch in `handle_event()`	Keep all cell changes inside `Game` so the win check and counters stay correct.
A new on-screen element	Add a `draw_*()` method in `ui.py`, call it from `render()`, and check its click area at the top of `handle_event()`	The mine selector, timer and record line all follow this pattern.
A different board size	`BoardManager.ROWS` and `COLS`, `BOARD_SIZE` in `game.py`, the `range(10)` in `mine_generator.py`, and the label lists in `draw_labels()`	The size is hard-coded in these four places today. Pointing them all at `BoardManager` is the first step.
A wider mine range	`MIN_MINES` and `MAX_MINES` in `game.py`	The limit is enforced by the selector in the UI, not by `Game`.
Records that persist between sessions	Replace `record_seconds` and `record_difficulty` in `MinesweeperUI` with a class that loads and saves a file	`record_win()` is the single place a record is set.
A different front end, such as a text or web UI	Write a new module that calls `Game` and reads `BoardManager`	`game.py`, `board.py`, `cell.py`, `ai_solver.py` and `game_timer.py` do not depend on pygame.

Only the following modules in `src/` have a matching test file in `tests/`: board.py, cell.py, game.py, and number_calculator.py. A change to one layer can be checked by running that layer's tests alone, and the UI tests run without opening a window.